when accessing an agent. Different aspects of security can be configured by the manager to allow message authentication, confidentiality, and integrity.

SNMPv3 also allows remote configuration of security aspects without requiring the administrator to actually be at the place where the device is located.

9.3 ASN.1

In data communication, when we send a continuous stream of bits to a destination, we somehow need to define the format of the data. If we send a *name* and a *number* in a single message, we need to tell the destination that, for example, the first 12 bits define the name and the next 8 bits define the number. We will have more difficulty when we send a complex data type such as an array or a record. For example, in the case of an array, if we send 2000 bits in a message, we need to tell the receiver that it is an array of 200 numbers each of 10 bits or it is an array of 10 numbers each of 200 bits.

A solution is that we separate the definition of data types from the sequence of bits transmitted through the network. This is done through an abstract language that uses some symbols, key words, and atomic data types and lets us make new data types out of the simple types. The language is called Abstract Syntax Notation One (ASN.1). Note that ASN.1 is a very complex language used in different areas of computer science, but in this section, we only introduce the language as much as needed for the SNMP protocol.

9.3.1 Language Basics

Before we show how we can define objects and associated values, let us talk about the language itself. The language use some symbols and some key words and defines some primitive data types. As we said before, SMI uses a subset of these entities in its own language.

Symbols

The language uses a set of symbols, given in Table 9.5. Some of these symbols are single characters, but some are pairs of characters.

Table 9.5 *Symbols used in ASN.1*

Symbol	*Meaning*	*Symbol*	*Meaning*
::=	Defined as or assignment	..	Range
|	Or, alternative, or option	{ }	Start and end of a list
–	Negative sign	[]	Start and end of tag
––	The following is a comment	()	Start and end of a subtype

Keywords

The language has a set of limited keywords that can be used. These words can be used in the language only for the purpose for which they have been defined. All words should be in uppercase (Table 9.6).

Table 9.6 *Keywords in ASN.1*

Keyword	*Description*
BEGIN	Start of a module
CHOICE	List of alternatives
DEFINITIONS	Definition of a data type or an object
END	End of a module
EXPORTS	Data type that can be exported to other modules
IDENTIFIER	A sequence of non-negative numbers that identifies an object
IMPORTS	Data type defined in an external module and imported
INTEGER	Any positive, zero, or negative integer
NULL	A null value
OBJECT	Used with IDENTIFIER to uniquely define an object
OCTET	Eight-bit binary data
OF	Used with SEQUENCE or SET
SEQUENCE	An ordered list
SEQUENCE OF	An ordered array of data of the same type
SET	An unordered list
SET OF	An array of unordered lists
STRING	A string of data

9.3.2 Data Types

After discussing the symbols and keywords used in the language, it is time to define its data types. The idea is similar to what we see in computer languages such as C, C++, or Java. In ASN.1, we have several simple data types such as integer, float, boolean, char, and so on. We can combine these data types to create a new simple data type (with a different name) or to define a structured data types such as array or struct. We first define simple data types in ASN.1 and then show how to make a new data type of these data types.

Simple Data Types

ASN.1 defines a set of simple (atomic) data types. Each data type is given a universal tag and has a set of values, as shown in Table 9.7. This is the same idea as used in a computer language when we have some basic data types with predefined ranges of values. For example, in C language, we have the data type *int,* which can take a range of values. Note that the tag in the table is actually the rightmost five bits of the tag we defined in Table 9.2.

Table 9.7 *Some simple ASN.1 built-in types*

Tag	***Type***	***Set of values***
Universal 1	BOOLEAN	TRUE or FALSE
Universal 2	INTEGER	Integers (positive, 0, or negative)
Universal 3	BIT STRING	A string of binary digits (bits) or a null set
Universal 4	OCTET STRING	A string of octets or a null set
Universal 5	NULL	Null, single valued
Universal 6	OBJECT IDENTIFIER	A set of values that defines an object
Universal 7	ObjectDescriptor	Human readable text describing an object

Table 9.7 *Some simple ASN.1 built-in types (continued)*

Tag	*Type*	*Set of values*
Universal 8	EXTERNAL	A type which is not in the standard
Universal 9	REAL	Real numbers in scientific notation
Universal 10	ENUMERATED	A list of integers
Universal 16	SEQUENCE, SEQUENCE OF	Ordered list of types
Universal 17	SET, SET OF	Unordered list of types
Universal 18	NumericString	Digits 0-9 and space
Universal 19	PrintableString	printable characters
Universal 26	VisibleString	ISO646String
Universal 27	GeneralString	General character string
Universal 30	CHARACTER STRING	Character set

New Data Types

ASN.1 uses **Backus–Naur Form** (**BNF**) syntax for defining a new data type from a built-in data type or a previously defined data type as shown below:

<new type> ::= **<type>**

where the *new type* must start with a capital letter.

Example 9.6

The following is an example of some new types using built-in types from Table 9.7.

```
Married ::= BOOLEAN
MaritalStatus ::= ENUMERATED {single, married, widowed, divorced}
DayOfWeek ::= ENUMERATED {sun, mon, tue, wed, thu, fri, sat}
Age ::= INTEGER
```

New Subtypes

ASN.1 even allows us to create a subtype whose range is a subrange of a built-in type or a previously defined data type.

Example 9.7

The following shows how we can make three new subtypes. The range of the first is the subset of INTEGER, the range of the second is the subset of REAL, and the range of the third is the subset of DayOfWeek, which we defined in Example 9.6. Note that we use the symbol (..) to define the range and the symbol (|) to define the choice.

```
NumberOfStudents ::= INTEGER (15..40)     -- An integer with the range 15 to 40
Grade ::= REAL (1.0..5.0)                 -- A real number with the range 1.0 to 5.0
Weekend ::= DayOfWeek (sun | sat)         -- A day that can be sun or sat
```

Simple Variables

In a programming language, we can create a variable of a particular type and assign (store) a value in it. In ASN.1, the term *Value Name* is used instead of variable, but we

use the term *variable,* which is more familiar to programmers. We can create a variable of a particular type and assign a value belonging to the range defined for that type. The following shows the syntax:

<variable> <type> ::= <value>

The name of the variable should start with a lower-case letter to distinguish it from the type.

Example 9.8

The following are a few examples of defining some variables and assigning the appropriate value from the range of those types. Note that the first and the third variables are of the built-in type, the second is of the type defined in Example 9.6, and the last is of a subtype defined in Example 9.7.

```
numberOfComputers INTEGER ::= 2
married Married ::= FALSE
herAge INTEGER ::= 35
classSize NumberOfStudents ::= 22
```

Structured Type

ASN.1 uses a keyword SEQUENCE to define a structured data type similar to *struct* (record) in C language or C++. The SEQUENCE type is an ordered list of variable types. The following shows a new type StudentAccount which is a sequence of three variables: username, password, and accountNumber.

```
StudentAccount ::= SEQUENCE
{
        userName VisibleString,
        password VisibleString,
        accountNumber INTEGER
}
```

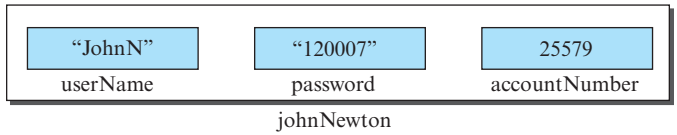
Structure Variables

After defining the new type, we can create a variable out of it and assign values to variables as shown below:

```
johnNewton StudentAccount
{
        userName "JohnN",
        password "120007",
        accountNumber 25579
}
```

Figure 9.23 shows the record created from the type definition and value assignments.

Figure 9.23 *Record representing the type definition and variable declaration*



We use the keyword SEQUENCE OF to define a new type similar to an array in C or C++ which is a composite type in which all components are the same. For example, we can define a forwarding table in a router as SEQUENCE OF Rows in which each Row is itself a sequence made of several variables.

9.3.3 Encoding

After the data has been defined and values are associated with variables, ASN.1 uses one of the encoding rules to encode the message to be sent. We already discussed the Basic Encoding Rule in the previous section.

9.4 END-CHAPTER MATERIALS

9.4.1 Further Reading

For more details about subjects discussed in this chapter, we recommend the following books and RFCs. The items enclosed in brackets refer to the reference list at the end of the book.

Books

Several books give thorough coverage of SNMP: [Com 06], [Ste 94], [Tan 03], and [MS 01]

RFCs

Several RFCs show different updates on SMTP, including RFC 3410, RFC 3412, RFC 3415, and RFC 3418. More information about MIB can be found in RFC 2578, RFC 2579, and RFC 2580.

9.4.2 Key Terms

Abstract Syntax Notation One (ASN.1)
Backus–Naur Form (BNF)
Basic Encoding Rules (BER)
Management Information Base (MIB)
object identifier
simple data type
Simple Network Management Protocol (SNMP)
Structure of Management Information (SMI)
structured data type
Trap

9.4.3 Summary

The five areas comprising network management are configuration management, fault management, performance management, accounting management, and security management. Configuration management is concerned with the physical or logical

changes of network entities. Fault management is concerned with the proper operation of each network component. Performance management is concerned with the monitoring and controlling of the network to ensure the network runs as efficiently as possible. Security management is concerned with controlling access to the network. Accounting management is concerned with controlling user access to network resources through charges.

Simple Network Management Protocol (SNMP) is a framework for managing devices in an internet using the TCP/IP protocol suite. A manager, usually a host, controls and monitors a set of agents, usually routers. SNMP uses the services of SMI and MIB. SMI names objects, defines the type of data that can be stored in an object, and encodes the data. MIB is a collection of groups of objects that can be managed by SNMP. MIB uses lexicographic ordering to manage its variables.

Abstract Syntax Notation Number One (ASN.1) is a language that defines the syntax and semantics of data. It uses some symbols, keywords, simple and structured data types. Part of ASN.1 is used by SMI to define the format of objects and values used in network management.

9.5 PRACTICE SET

9.5.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check his/her understanding of the materials before continuing with the practice set.

9.5.2 Questions

Q9-1. A network manager decides to replace the old router that connects the organization to the Internet with a more powerful one. What area of network management is involved here?

Q9-2. A network manager decides to replace a version of accounting software with a new version. What area of network management is involved here?

Q9-3. Distinguish between reactive fault management and proactive fault management.

Q9-4. If network management does not replace a component whose lifetime has been expired, what area in network management has been ignored?

Q9-5. Which of the following is not one of the five areas of network management defined by ISO?

a. fault **b.** performance **c.** personnel

Q9-6. Which of the following is not part of configuration management?

a. reconfiguration **b.** encryption **c.** documentation

Q9-7. Which of the following devices cannot be a manager station in SNMP?

a. a router **b.** a host **c.** a switch

Q9-8. Does an SNMP manager run a client SNMP program or a server SNMP program?

Q9-9. Distinguish between internal and external data traffic in an organization.

Q9-10. If a student in a college can monopolize access to a piece of software, causing other students to wait for a long time, which area of network management has failed?

Q9-11. Find the type (simple, sequence, sequence of) of the following objects in SMI.

a. An unsigned integer

b. An IP address

c. An object name

d. A list of integers

e. A record defining an object name, an IP address, and an integer

f. A list of records in which each record is an object name followed by a counter.

Q9-12. What is the length of the value field in the following BER encoding?

04 09 48 65 6C 4C …

Q9-13. Show how the textual name "iso.org.dod" is numerically encoded in SMI.

Q9-14. Is it possible to have a textual name in SMI as "iso.org.internet"? Explain.

Q9-15. Can an SNMP message reference a leaf node in the MIB tree? Explain.

Q9-16. Assume that a manageable object has only three simple variables. How many leaves can be found in the MIB tree for this object?

Q9-17. Distinguish between SMI and MIB.

Q9-18. What does the ***if*** object in MIB define? Why does this object need to be managed?

Q9-19. Assume that an object identifier in MIB has three simple variables. If the object identifier is *x*, what is the identifier of each variable?

Q9-20. Can SNMP reference the entire row of a table? In other words, can SNMP retrieve or change the values in the entire row of a table? Explain.

Q9-21. In SNMP, which of the following PDUs are sent from a client SNMP to a server SNMP?

a. GetRequest **b.** Response **c.** Trap

Q9-22. What are the source and destination port numbers when an SNMP message carries one of the following PDUs?

a. GetRequest **b.** Response **c.** Trap **d.** Report

Q9-23. Assume that a manageable object has a table with three columns. How many leaves are there in the MIB tree for this table?

Q9-24. Distinguish between a GetRequest PDU and a SetRequest PDU.

9.5.3 Problems

P9-1. Assume that an object *x* has two simple variables and one table with two columns. What is the identifier for each variable and each column of the table? We assume that simple variables come before the table.

P9-2. Assume that an object *x* has one simple variable and two tables with two and three columns, respectively. What is the identifier for the variable and each column of each table? We assume that the simple variable comes before the tables.

P9-3. Assume that an object *x* has two simple variables: an integer and an IP address. What is the identifier for each variable?

P9-4. Assume that an object *x* has only one table with two columns. What is the identifier for each column?

P9-5. One of the objects (groups) that can be managed is the ***ip*** group with the object identifier (1.3.6.1.2.1.**4**) in which (1.3.6.1.2.1) is the identifier of MIB-2 and (4) defines the ***ip*** group. In an agent, this object has 20 simple variables and three tables. One of the tables is the routing (forwarding) table with the identifier (1.3.6.1.2.1.**4**.**21**). This table has eleven columns, the first of which is called the *ipRouteDes,* which means the destination IP address. Assume that the indexing is based on the first column. Assume that the table has four rows at the moment with the destination IP addresses (201.14.67.0), (123.16.0.0), (11.0.0.0), and (0.0.0.0). Show how SNMP can access all four instances of the second column, called *ipRouteIfIndex,* which defines the interface numbers through which the IP should be sent out.

P9-6. Show the encoding for the INTEGER 1456 using BER.

P9-7. Show the encoding for the OCTET STRING "Hello world." using BER.

P9-8. Show the encoding for the IPAddress 112.56.23.78 using BER.

P9-9. Object *x* has two simple variables. How can SNMP refer to the instance of each variable?

P9-10. Object *x* has one table with two columns. The table at this moment has three rows with the contents shown below. If the table index is based on the values in the first column, show how SNMP can access each instance.

Object *x*

a	aa
b	bb
c	cc

Table

P9-11. Assume that a data structure is made of an INTEGER of value (131) and another structure made of an IPAddress of value (24.70.6.14) and an OCTET-STRING ("UDP"). Using BER, encode the data structure.

P9-12. Given the code `02040000C738`, decode it using BER.

P9-13. Given the code `300C02040000099806040A05030E`, decode it using BER.

P9-14. Given the code `300D04024E6F300706030103060500`, decode it using BER.

P9-15. Show the encoding for the object identifier 1.3.6.1.2.1.7.1 (the udpInDatagram variable in udp group) using BER.

P9-16. Using BER, show how we can encode a structured data type made of an INTEGER of value (2371), an OCTET STRING of value ("Computer"), and an IPAddress of value (185.32.1.5) as shown below:

```
SEQUENCE
{
        INTEGER 2371
        OCTET STRING "Computer"
        IP Address 185.32.1.5
}
```

P9-17. Define a GetRequest PDU (*see* Figure 9.18) using the syntax defined for structured data types in ASN.1.

P9-18. Define a Response PDU (*see* Figure 9.18) using the syntax defined for structured data types in ASN.1.

P9-19. Assume that a manager needs to know the number of user datagrams an agent has sent out (a udpOutDatagrams counter with the identifier 1.3.6.1.2.1.7.4). Show the code for a Varbind that is sent in a GetRequest message and the code that the agent will send in the Response message if the value of the counter at this moment is 15.

P9-20. Define an SNMP message (*see* Figure 9.19) using the syntax defined for structured data types in ASN.1.

P9-21. Define a VarbindList (*see* Figure 9.19) using the syntax defined for structured data types in ASN.1.